

03a — Synchronous Composition

Lock-Step Parallel Execution

Mariano Cerrutti

Mununu Tutorial

Producer Automaton

```
1      automaton Producer {
2          controllable {
3              label produce;
4          }
5
6          states {
7              state Empty initial;
8              state DataReady;
9          }
10
11         transitions {
12             // Produce data
13             transition Empty -> DataReady on label produce;
14             // Transfer data to consumer (shared label --must sync)
15             transition DataReady -> Empty on label transfer;
16             // Skip: producer does nothing this cycle
17             transition Empty -> Empty on label p_skip;
18             transition DataReady -> DataReady on label p_skip;
```

Consumer Automaton

```
1      automaton Consumer {
2          controllable {
3              label consume;
4          }
5
6          states {
7              state Waiting initial;
8              state Processing;
9          }
10
11         transitions {
12             // Receive data via transfer (shared label --must sync)
13             transition Waiting -> Processing on label transfer;
14             // Finish processing
15             transition Processing -> Waiting on label consume;
16             // Skip: consumer does nothing this cycle
17             transition Waiting -> Waiting on label c_skip;
18             transition Processing -> Processing on label c_skip;
```

Composition Block

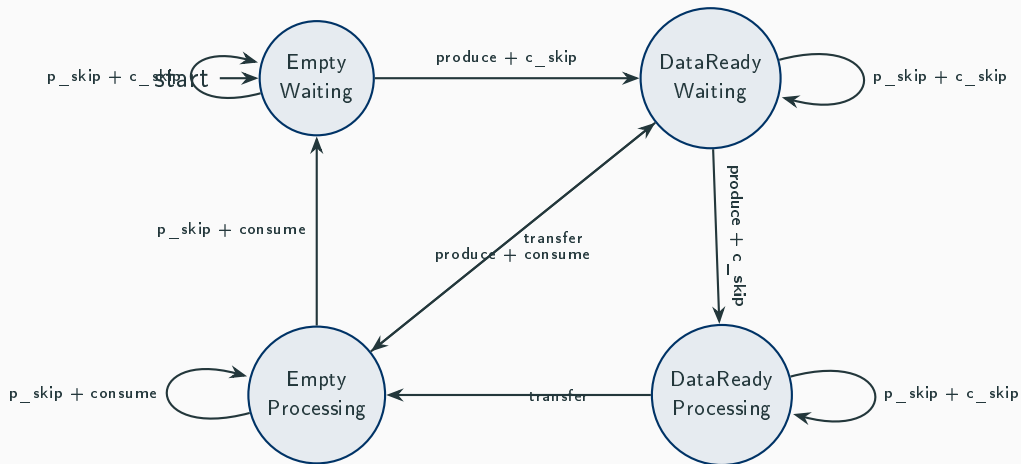
```
1  composition {  
2      synchronous pipeline {  
3          members [Producer, Consumer];  
4      }  
5  }
```

- `synchronous` — both automata step **simultaneously** every cycle.
- `members [Producer, Consumer]` — lists the participating automata.
- `pipeline` — the name of the composed system (used in `over`).

Rule	Description
Shared labels	A label present in <i>both</i> alphabets (e.g. <code>transfer</code>) must fire in both automata simultaneously.
Non-shared labels	Labels unique to one automaton (e.g. <code>produce</code> , <code>consume</code>) combine freely with any transition of the other side.
Skip labels	Each automaton needs its own skip (<code>p_skip</code> , <code>c_skip</code>) so it can “do nothing” while the other acts. A shared <code>idle</code> would force both to idle together.

Product State Space

The composed system pipeline has $2 \times 2 = 4$ states:



Formulas Over Compositions

```
1  mu_formulas {
2      // Safety over the composed system (structural --no state names needed)
3      formula safety_invariant {
4          over pipeline;
5          body = nu X. ([] X);
6      }
7
8      // No deadlocks: every reachable state has a successor
9      formula no_deadlock {
10         over pipeline;
11         body = nu X. ((<> true) && ([] X));
12     }
13
14     // Properties over individual automata (state predicates work here)
15     formula producer_can_produce {
16         over Producer;
17         body = mu X. (DataReady || <> X);
18     }
```

New Syntax Summary

Keyword	Purpose
<code>composition</code>	Block declaring composed systems
<code>synchronous</code>	Lock-step composition mode
<code>members</code>	Lists the automata to compose

Key takeaway: Synchronous composition models systems where every component takes a step on every clock cycle. Shared labels enforce coordination; private skip labels allow independent idling.